complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() map → reduce → store

filter() null checks + collect() map → reduce → store

filter() comparisons + collect() map → reduce → store

filter() method calls returning boolean + collect() map → reduce → store

distinct() rely on equals() / hashcode() + collect() map → reduce → store

distinct() immutable objects + collect() map → reduce → store

map() transform objects + collect() map → reduce → store

map() create new objects + collect() map → reduce → store

map() call pure functions + collect() map → reduce → store

flatmap() return stream + collect() map → reduce → store

flatmap() split data + collect() map → reduce → store

flatmap() expand collections + collect() map → reduce → store

sorted() natural ordering + collect() map → reduce → store

sorted() custom comparator + collect() map → reduce → store

sorted() multi-field compare + collect() map → reduce → store

peek() logging + collect() map → reduce → store

peek() debugging + collect() map → reduce → store

peek() metrics + collect() map → reduce → store

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** where **each pipeline explicitly follows**:

Map → Reduce → Store

- **Map** = transform data
- **Reduce** = aggregate (sum / count / max / min / join / etc.)
- **Store** = collect() into a Map or List

All examples:

- Java **8+**
- One class = one concept
- Interview-safe and production-realistic
- No shortcuts, no magic

---

1 filter() boolean checks + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterBooleanMapReduceStore {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(10, 15, 20, 25);
      Map<String, Integer> result = new HashMap<String, Integer>();

      for (Integer n : numbers) {

         if (n % 2 == 0) {                                    // FILTER

            int mapped = n * 2;                               // MAP

            String key = (mapped > 30) ? "HIGH" : "LOW";

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                    // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

2 filter() null checks + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterNullMapReduceStore {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("Anurag", null, "Amit", null);
      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (String name : names) {

         if (name != null) {                                    // FILTER

            String mapped = name.toUpperCase();                 // MAP
            Integer key = mapped.length();

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                         // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

3 filter() comparisons + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterComparisonMapReduceStore {
   public static void main(String[] args) {

      List<Integer> salaries = Arrays.asList(20000, 40000, 60000);
      Map<String, Integer> result = new HashMap<String, Integer>();

      for (Integer s : salaries) {

         if (s >= 30000) {                                    // FILTER

            int mapped = s + 5000;                            // MAP
            String key = (mapped >= 50000) ? "SENIOR" : "MID";

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                    // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

4 filter() method call boolean + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterMethodMapReduceStore {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {

      List<User> users = Arrays.asList(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
      );

      Map<Character, Long> result = new HashMap<Character, Long>();

      for (User u : users) {

         if (u.isActive()) {                                    // FILTER

            String mapped = u.name;                             // MAP
            Character key = mapped.charAt(0);

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                         // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

5 distinct() equals/hashCode + collect() (Map → Reduce → Store)

```
import java.util.*;

public class DistinctEqualsMapReduceStore {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (o instanceof Person) {
            Person p = (Person) o;
            return name.equals(p.name);
         }
         return false;
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> people = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> distinct = new HashSet<Person>(people);

      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (Person p : distinct) {

         String mapped = p.name;                          // MAP
         Integer key = mapped.length();

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                      // REDUCE + STORE
      }

      System.out.println(result);
```

```
    }
}
```

---

6 distinct() immutable objects + collect() (Map → Reduce → Store)

```
import java.util.*;

public class DistinctImmutableMapReduceStore {

    public static void main(String[] args) {

        // Java 1.5 way to create list
        List<String> values = new ArrayList<String>();
        values.add("Java");
        values.add("Java");
        values.add("Go");
        values.add("Python");

        // DISTINCT step
        Set<String> distinctSet = new HashSet<String>(values);

        // MAP + REDUCE + STORE
        Map<Character, Long> result = new HashMap<Character, Long>();

        for (String value : distinctSet) {

            // MAP (toUpperCase)
            String upper = value.toUpperCase();

            // key = first character
            Character key = upper.charAt(0);

            // REDUCE (counting)
            Long count = result.get(key);

            if (count == null) {
                result.put(key, 1L);
            } else {
                result.put(key, count + 1);
            }
        }

        System.out.println(result);
    }
}
```

---

7 map() transform objects + collect() (Map → Reduce → Store)

```
import java.util.*;

public class MapTransformMapReduceStore {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("anurag", "amit", "ravi");
      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (String name : names) {

         String mapped = name.toUpperCase();                    // MAP
         Integer key = mapped.length();

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                            // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

8 map() create new objects + collect() (Map → Reduce → Store)

```
import java.util.*;

public class MapNewObjectMapReduceStore {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> names = Arrays.asList("Anurag", "Amit", "Ravi");
      Map<Character, Long> result = new HashMap<Character, Long>();

      for (String name : names) {

         UserDTO dto = new UserDTO(name);                    // MAP
         Character key = dto.name.charAt(0);

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                         // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

9 map() pure functions + collect() (Map → Reduce → Store)

```
import java.util.*;

public class MapPureFunctionMapReduceStore {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(2, 3, 4);
      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : numbers) {

         int mapped = square(n);                          // MAP
         Boolean key = (mapped > 10);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                   // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

10 flatMap() return Stream + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FlatMapStreamMapReduceStore {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();
      lists.add(Arrays.asList(1, 2));
      lists.add(Arrays.asList(3, 4));

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (List<Integer> inner : lists) {                    // FLATTEN
         for (Integer n : inner) {

            int mapped = n * 10;                             // MAP
            Boolean key = (mapped > 20);

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                   // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

11 flatMap() split data + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FlatMapSplitMapReduceStore {
   public static void main(String[] args) {

      List<String> sentences = Arrays.asList("Java Stream", "Flat Map");
      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (String sentence : sentences) {

         String[] words = sentence.split(" ");                    // SPLIT

         for (int i = 0; i < words.length; i++) {

            String mapped = words[i].toLowerCase();               // MAP
            Integer key = mapped.length();

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                           // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

12 flatMap() expand collections + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FlatMapExpandMapReduceStore {
   public static void main(String[] args) {

      Map<String, List<String>> data = new HashMap<String, List<String>>();
      data.put("A", Arrays.asList("x", "y"));
      data.put("B", Arrays.asList("z"));

      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (List<String> list : data.values()) {                    // FLATTEN
         for (String value : list) {

            String mapped = value.toUpperCase();                    // MAP
            Integer key = mapped.length();

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                             // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

13 sorted() natural order + collect() (Map → Reduce → Store)

```
import java.util.*;

public class SortedNaturalMapReduceStore {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>(
            Arrays.asList(5, 1, 3));

      Collections.sort(numbers);                                  // SORT

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : numbers) {

         int mapped = n * 2;                                      // MAP
         Boolean key = (mapped > 5);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                           // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

14 sorted() custom comparator + collect() (Map → Reduce → Store)

```
import java.util.*;

public class SortedCustomMapReduceStore {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(names, Collections.reverseOrder());    // CUSTOM SORT

      Map<Character, Long> result = new HashMap<Character, Long>();

      for (String name : names) {

         String mapped = name.toUpperCase();                  // MAP
         Character key = mapped.charAt(0);

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                          // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

15 sorted() multi-field + collect() (Map → Reduce → Store)

```
import java.util.*;

public class SortedMultiFieldMapReduceStore {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> employees = new ArrayList<Employee>();
      employees.add(new Employee("Anurag", 30));
      employees.add(new Employee("Anurag", 25));
      employees.add(new Employee("Ravi", 35));

      Collections.sort(employees, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {

            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) return nameCompare;

            return e1.age - e2.age;
         }
      });

      Map<String, Integer> result = new HashMap<String, Integer>();

      for (Employee e : employees) {

         int mapped = e.age;                                    // MAP
         String key = (mapped >= 30) ? "SENIOR" : "JUNIOR";

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                         // REDUCE + STORE
      }

      System.out.println(result);
   }
```

```
}
```

---

16 peek() logging + collect() (Map → Reduce → Store)

```
import java.util.*;

public class PeekLoggingMapReduceStore {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);
      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : list) {

         System.out.println("Log: " + n);                    // PEEK

         int mapped = n * 10;                                // MAP
         Boolean key = (mapped > 20);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                      // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

17 peek() debugging + collect() (Map → Reduce → Store)

```
import java.util.*;

public class PeekDebugMapReduceStore {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);
      Map<Boolean, Long> result = new HashMap<Boolean, Long>();

      for (Integer n : list) {

         if (n == 20) {
            System.out.println("Debug hit: " + n);                    // PEEK
         }

         int mapped = n + 1;                                          // MAP
         Boolean key = (mapped > 20);

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                                  // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

18 peek() metrics + collect() (Map → Reduce → Store)

```
import java.util.*;

public class PeekMetricsMapReduceStore {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);
      int counter = 0;

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : list) {

         counter++;                                          // METRICS (peek equivalent)

         int mapped = n * 2;                                 // MAP
         Boolean key = (mapped > 20);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                      // REDUCE + STORE
      }

      System.out.println("Processed: " + counter);
      System.out.println(result);
   }
}
```